Name: NGO TUNG CHUONG

ID: s225586848

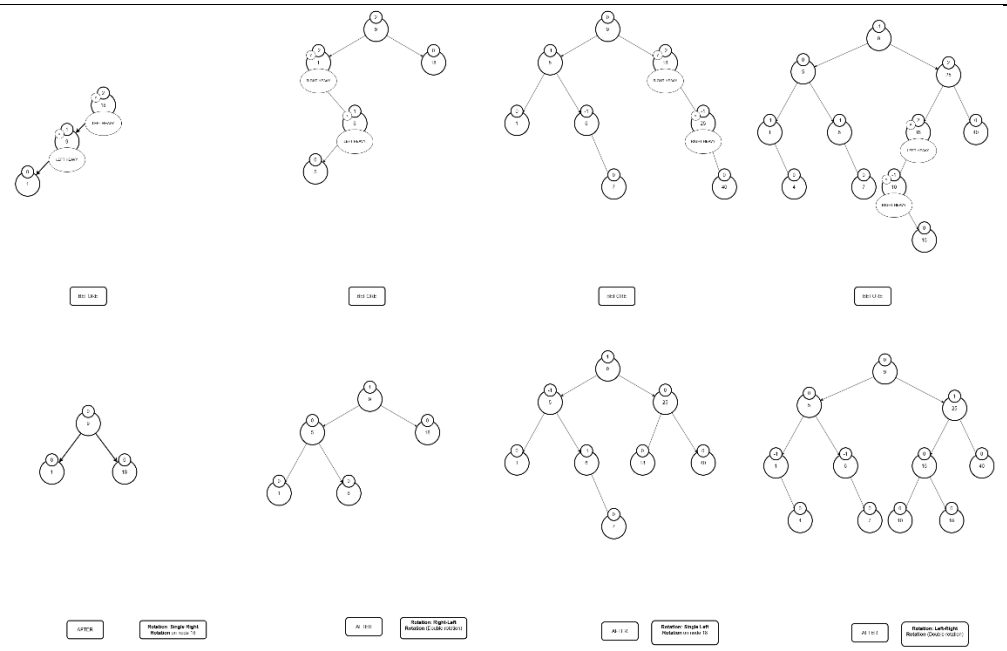
GitHub: https://github.com/NoXxChapter/SIT221-DataStructure.git

1. Draw a series of figures demonstrating the insertion of the values

   18, 9, 1, 6, 5, 7, 25, 40, 10, 4, 16

   into an initially ***empty*** AVL tree. Insert the values in the order they appear in the given sequence. You must:

   - Show the resulting AVL tree immediately before and after each insertion step that causes the tree's rebalancing.
   - Calculate the ***balance degree*** (as the difference between the heights of the left and the right subtrees rooted at a node) and label each node of the AVL tree before and after the necessary rebalancing.
   - Clearly indicate the node(s) at which the rotation is performed. Here, let v be the first node you encounter in going up from the newly added node, say z, towards the root of the AVL tree T such that v is unbalanced. Then, let x be the child of v such that x is the ancestor of z. Determine whether the subtree of T rooted at v is ***right*** or ***left heavy***. Similarly, indicate whether the subtree of T rooted at x is ***right*** or ***left heavy***.
   - Performing a ***single*** or a ***double rotation*** as a rebalancing (repairing) operation on the AVL tree, specify the type of the rotation that you apply: ***Single Left Rotation***, ***Single Right Rotation***, ***Left-Right Rotation***, or ***Right-Left Rotation***.
   - Each time a new value is added, ensure that both the ***Binary Search Tree Property*** and the ***Height-Balance (AVL Tree) Property*** are maintained.



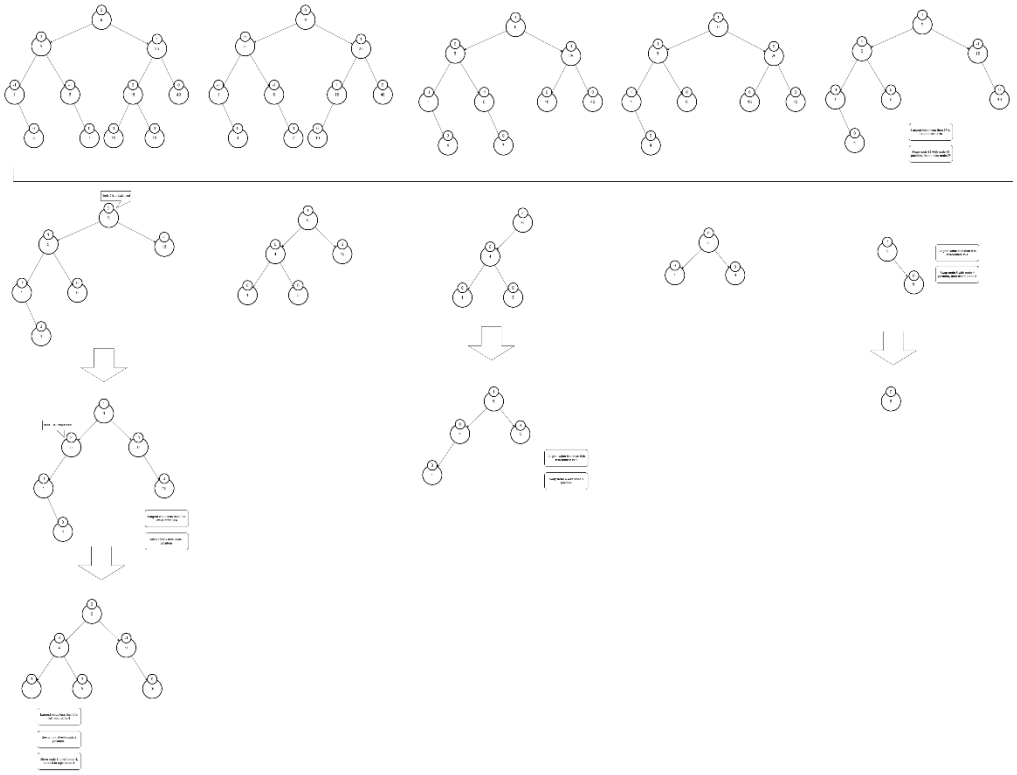
(please download the pic from GitHub, it better to view)

2. Now, draw a series of figures showing the deletion of the values:

18, 10, 7, 25, 40, 9, 16, 4, 5, 1, 6

from the AVL tree built in the previous part of the task. Delete the values in the order they appear in the given sequence. Draw the AVL tree after each deletion and rotation (if any) and complement each of the figures with all the aforementioned details.

When you delete a node with two children, you must always replace it with ***'the largest element smaller than the key of the deleted node'*** (as opposed to the other possible rule: 'the smallest element larger than the key of the deleted node'). **Following this rule is important** to let us check your solution quickly.



(please download the pic from GitHub, it better to view)

Task 3

Develop a linear $O(n)$ time algorithm that accepts an arbitrary binary search tree as an input and creates a **balanced** binary search tree whose height is $O(\log n)$. As the answer to this question, provide its pseudocode and description. Explain why you believe that your algorithm takes a linear time.

```
getInorder(node, list) {
        If node is null
            return;
        getInorder(node.left, list);
        list.add(node.data);
        getInorder(node.right, list);
}

balancedBST(list, start, end) {
        if start > end
            return null;
```

```
        mid = (start + end) / 2;
        newNode = new Node(list[mid]);

        newNode.left = balancedBST(list, start, mid – 1);
        newNode.right = balancedBST(list, mid + 1, end);
        return newNode;
}
```

Performce an inorder traverse to extract all elements from the unsorted BST into a sorted array or list. By the time, to ensure the new tree is balanced $Olog(n)$ height, we find the middle element of the sorted list. Then later, we utilize the recursive to repeat for each other half of the list to create left and right subtrees.

The time complexity:

When we extracted all elements from unsorted BST. It only traverses each element once, so it takes $0(n)$ time. Also, with the "building" process, each insertion takes up only $O(1)$ each element. So, it also takes up to $O(n)$ time. But by always picking the median as the root, the height of the resulting tree is guaranteed to be $Olog(n)$ satisfying the balance requirement.